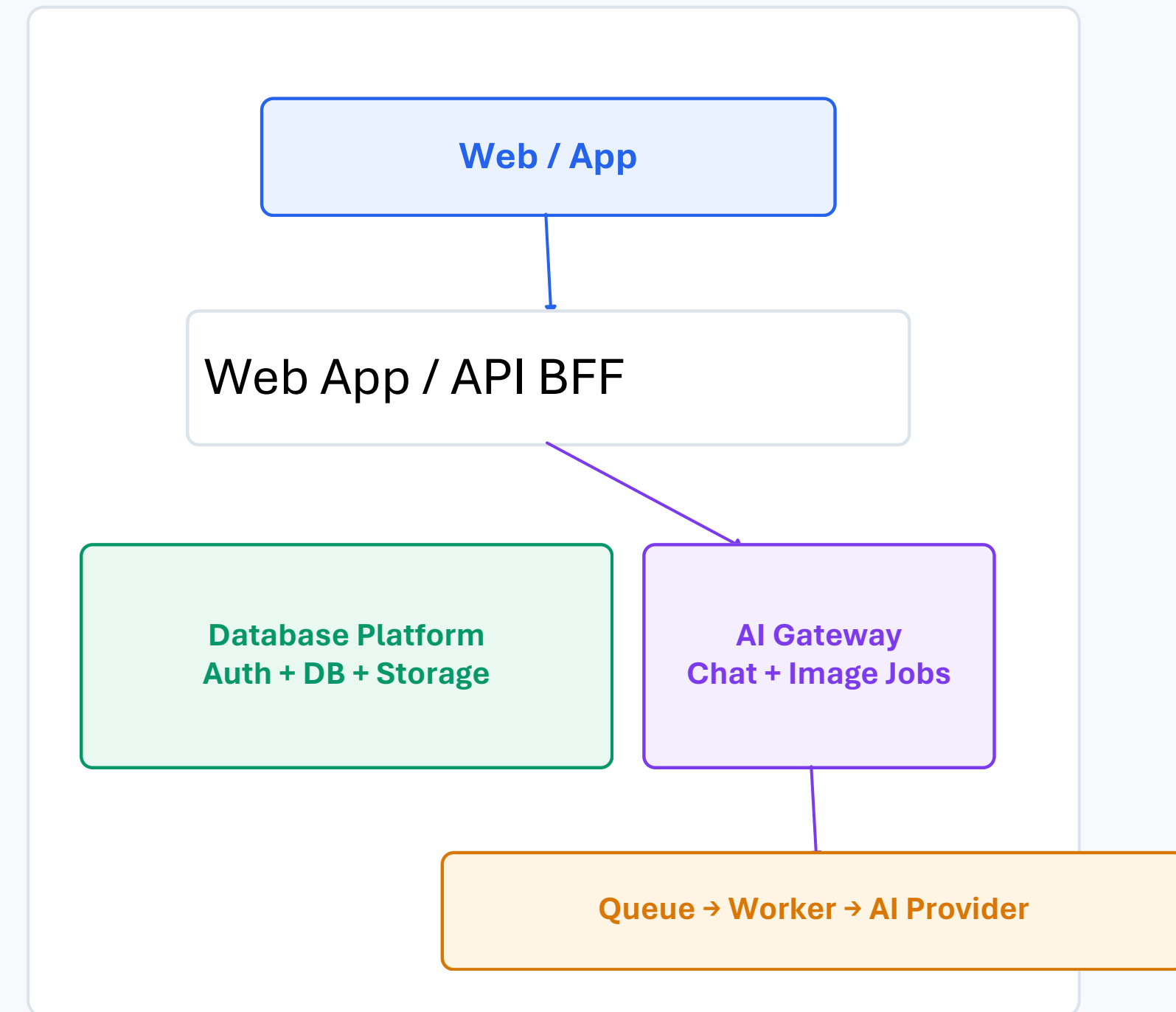


DooDee Scaling Architecture

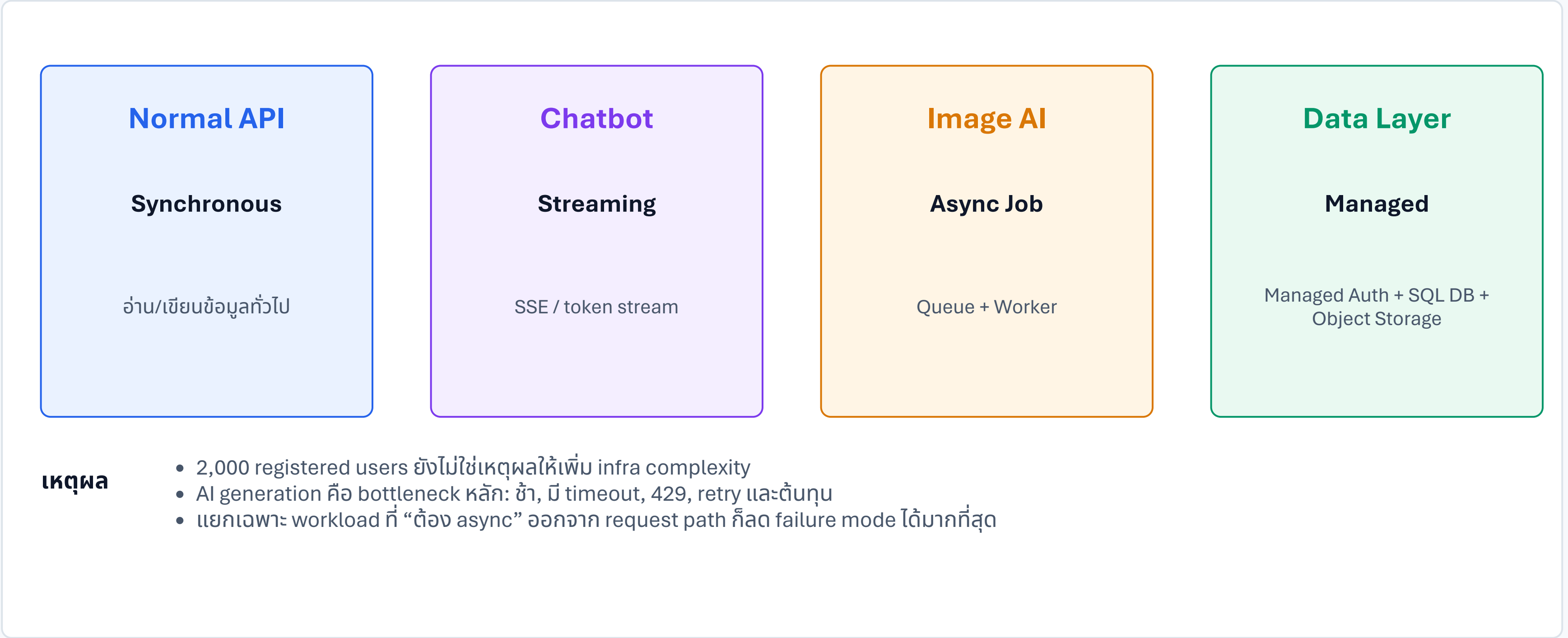
โครงสร้างที่ควรใช้ตอนนี้สำหรับ ~2,000 users
AI image generation + treatment preview + chatbot •
Database/Storage vendor-agnostic

Design goal: ไม่พังง่าย • ไม่แลคเพราะ AI • คุมค่า API ได้ • scale ที่ละขั้น



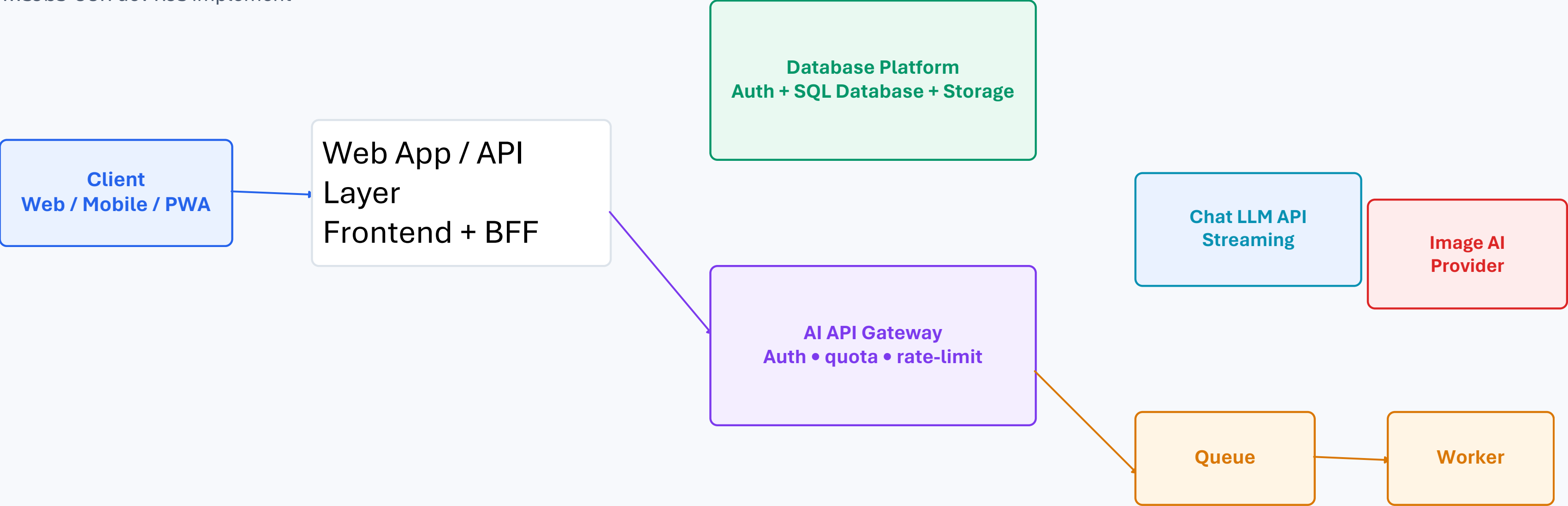
1. Architecture decision

ใช้ Modular Monolith + Async AI Jobs ก่อน — ยังไม่ต้อง microservices / Kubernetes



2. Target architecture

ภาพรวมระบบที่ dev ควร implement



Principle

Web request ต้องไม่ “รอ image generation จนเสร็จ” — ให้ request จบหลังสร้าง job แล้วค่อยประมวลผลเบื้องหลัง

3. Image generation pipeline

จุดสำคัญที่สุดของ DooDee ตอนนี้



Required job states

uploading

queued

processing

completed

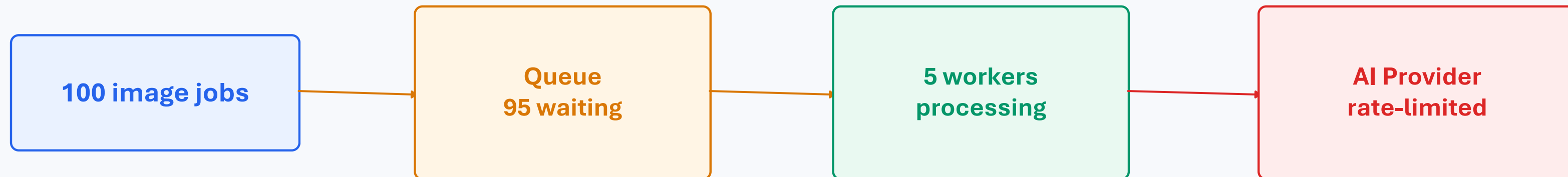
failed

cancelled

Browser refresh แล้วต้อง recover job ได้จาก DB — ห้ามผูกสถานะไว้กับ React memory อย่างเดียว

4. Queue + Worker = backpressure

traffic spike ต้องทำให้ “คิวยาวขึ้น” ไม่ใช่ “ทั้งระบบล่ม”



- เริ่ม WORKER_CONCURRENCY ต่ำ เช่น 3–5 แล้วปรับตาม provider limits + latency + budget
- อย่า autoscale แบบไม่มีเพดาน เพราะ scale = ค่า API เพิ่มทันที
- queue depth + oldest queued job ต้องขึ้น monitoring dashboard

5. Chatbot flow

chat ต้อง synchronous + streaming — ไม่เข้า queue แบบ image generation



Context strategy System Prompt + User/Face Context + Conversation Summary + Recent Messages + Current Message

เมื่อ thread ยาว: summarize old messages แล้วเก็บ summary แทนการส่ง history ทั้งหมดทุกครั้ง → latency และ token cost ลดลง

6. Core database schema

schema ขั้นต่ำที่ต้องมีเพื่อ trace งาน AI และคุมต้นทุน

ai_jobs

- id / user_id / type
- status / provider / model
- input_path / output_path
- idempotency_key
- attempt_count / error_code
- actual_cost / timestamps

chat_threads

- id / user_id / title
- summary
- created_at / updated_at
- last_message_at

chat_messages

- id / thread_id / role
- content / model
- input_tokens / output_tokens
- latency_ms / created_at

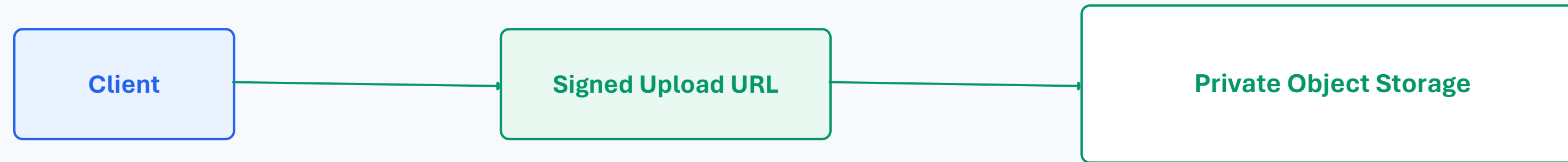
usage_ledger

- id / user_id / feature
- job_id / units / cost
- reserved / settled / refunded
- created_at

Critical constraint: UNIQUE(user_id, idempotency_key)

7. Face image storage

รูปใบหน้าคือข้อมูลสำคัญ — storage ต้อง private และ upload ตรง



อย่า proxy file bytes ผ่าน Next.js function โดยไม่จำเป็น

อย่าเก็บ base64 image ใน SQL Database

Path: /users/{user_id}/generations/{uuid}/input.webp และ output.webp

Serve ด้วย signed URL / presigned URL; สิทธิ์ต้องผูกกับ user / clinic access policy

8. Reliability controls

สิ่งที่หยุด duplicate cost และ provider failure ไม่ให้ลามเข้าระบบ

Idempotency	MUST	double tap / retry ต้องไม่สร้างรูปซ้ำ
Rate limit	MUST	user + IP + global provider concurrency
Retry	CONTROLLED	เฉพาะ 429 / 5xx / network พร้อม backoff + jitter
No blind retry	MUST	timeout หลัง provider ใช้งานแล้วอาจทำให้ charge ซ้ำ
Provider abstraction	SHOULD	เปลี่ยน model/provider โดยไม่แตะ business logic

AIProvider interface: generateImage() • generateText() • checkStatus() • estimateCost()

9. Database performance

ป้องกัน DB พังก่อนจะถึงเรื่อง scale ใหญ่

Connection Pooling

Vercel Functions

DB Connection Pool

SQL Database

Indexes

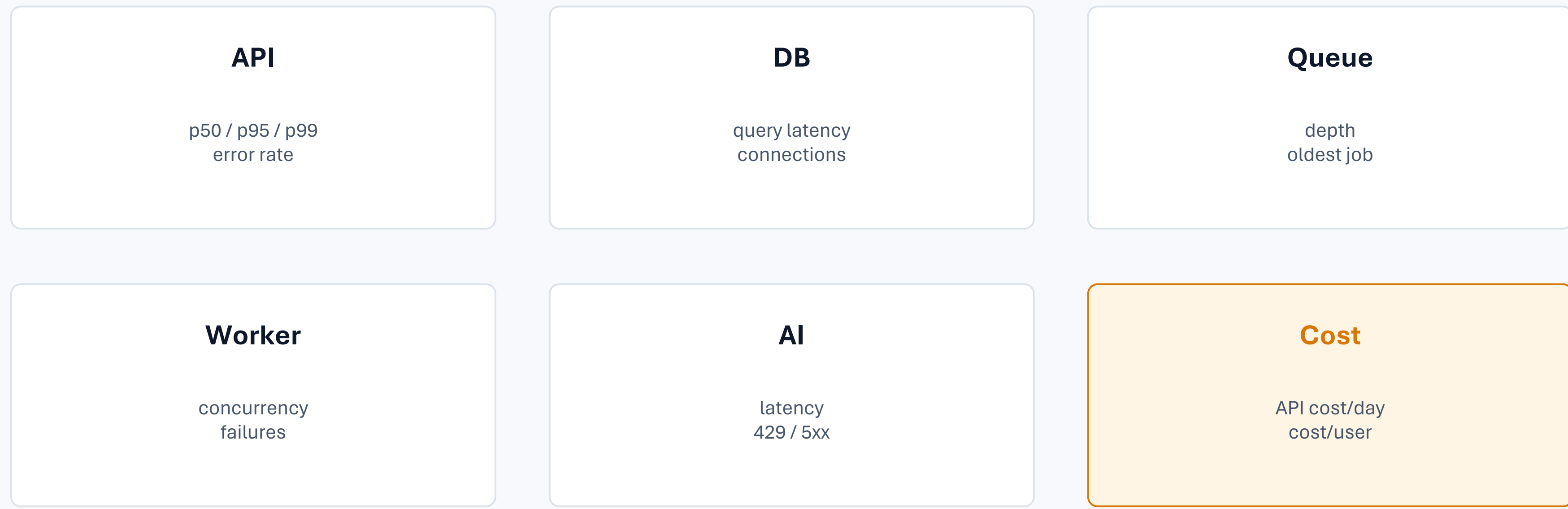
- ai_jobs(user_id, created_at DESC)
- ai_jobs(status, created_at)
- chat_messages(thread_id, created_at)
- usage_ledger(user_id, created_at)
- partial index: queued / processing

Query discipline

- หลีกเลียง N+1 queries
- ใช้ pagination ทุก list ใหญ่
- วัด p95 query latency
- ใช้ EXPLAIN ก่อนเพิ่ม index
- อย่า index ทุก column

10. Monitoring / Observability

ถ้าระบบช้า ต้องตอบได้ว่า “ช้าตรงไหน” ภายในไม่กี่นาที



Dashboard ที่ต้องเห็นเด่นที่สุด: AI_COST_TODAY + Queue Oldest Job + Provider 429

11. Initial SLO + load test

ตัวเลขเหล่านี้คือ engineering targets สำหรับเริ่มต้น — ต้องปรับตาม traffic จริง

Initial SLO

Normal API p95	< 500 ms
Create image job p95	< 500 ms
Error rate	< 1%
Return job_id	< 1 sec
Duplicate generation	0

Load test scenarios

- 100 concurrent normal users
- 30 concurrent chatbot streams
- 20 simultaneous image submissions
- 100 image jobs queued
- Provider returns 429 / timeout
- Worker crashes mid-job
- DB latency spike

12. Dev implementation priority

ถ้าจะเริ่มพรุ่งนี้ ให้เดินตามลำดับนี้

P0 — ทำก่อน

- AI Job table
- Queue + Worker
- Idempotency
- Rate limit
- Private Storage + direct upload
- Usage ledger
- Error handling

P1 — ต่อทันที

- DB indexes
- Connection pooling
- Chat streaming
- Chat context summary
- Observability
- Cost tracking

P2 — เมื่อมีข้อมูล

- Redis cache
- Provider fallback
- Worker autoscaling
- Advanced alerts

13. What NOT to build yet

อย่าใช้ dev time กับ infra ที่ยังไม่แก้ bottleneck จริง



Definition of Done — Phase 1

User submit image → job created <1s → queue → worker → provider → private storage → completed/failed status → UI recovers after refresh → cost recorded → no duplicate charge

Start here: Image generation pipeline first. It is the slowest, most expensive, and highest-risk path in DooDee.